

LAB EXPERIMENT

NAME : Narendra K

REG NO : 192311236

COURSE CODE: (CSA0618)

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHM.

Experiment 31: Sequential Search – Find an Element

Aim

To implement Sequential Search to find a given element in an array.

Algorithm

1. Read the array and the search key.
2. Start searching from the first element.
3. Compare each element with the key.
4. If the key is found, display its position.
5. Otherwise, display "Element not found".
6. Count the number of comparisons.

Program

```
def sequential_search(arr, key):  
    comparisons = 0  
    for i in range(len(arr)):  
        comparisons += 1  
        if arr[i] == key:  
            return i + 1, comparisons  
    return -1, comparisons
```

```
arr = [12, 25, 8, 45, 32, 19, 50]
key = 32

pos, comp = sequential_search(arr, key)

if pos != -1:
    print("Element found at position", pos)
else:
    print("Element not found")

print("Number of comparisons =", comp)
```

Sample Input

Array: 12 25 8 45 32 19 50

Key: 32

Sample Output

Element found at position 5

Number of comparisons = 5

Result

Thus, the given element was found successfully using Sequential Search.

Experiment 32: Sequential Search – Unsuccessful Search

Aim

To implement Sequential Search for an unsuccessful search.

Algorithm

1. Read the array and key.
2. Compare every element with the key.

3. If found, display its position.
4. Otherwise display "Element not found".
5. Display the total comparisons.

Program

```
def sequential_search(arr, key):  
    comparisons = 0  
  
    for i in range(len(arr)):  
        comparisons += 1  
        if arr[i] == key:  
            return i + 1, comparisons  
  
    return -1, comparisons  
  
arr = [5, 10, 15, 20, 25, 30, 35]  
key = 18  
  
pos, comp = sequential_search(arr, key)  
  
if pos == -1:  
    print("Element not found")  
else:  
    print("Element found at position", pos)  
  
print("Number of comparisons =", comp)
```

Sample Input

Array: 5 10 15 20 25 30 35

Key: 18

Sample Output

Element not found

Number of comparisons = 7

Result

Thus, the unsuccessful Sequential Search was implemented successfully.

Experiment 33: Sequential Search – First Occurrence

Aim

To find the first occurrence of an element using Sequential Search.

Algorithm

1. Read the array and key.
2. Search from left to right.
3. Stop when the first occurrence is found.
4. Display its position.

Program

```
def first_occurrence(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i + 1  
    return -1
```

```
arr = [10, 25, 15, 25, 30, 25, 40]
```

```
key = 25
```

```
pos = first_occurrence(arr, key)

if pos != -1:
    print("First occurrence at position", pos)
else:
    print("Element not found")
```

Sample Input

Array: 10 25 15 25 30 25 40

Key: 25

Sample Output

First occurrence at position 2

Result

Thus, the first occurrence of the element was found successfully.

Experiment 34: Sequential Search – All Occurrences**Aim**

To find all occurrences of a given element using Sequential Search.

Algorithm

1. Read the array and key.
2. Compare each element with the key.
3. Store all matching positions.
4. Display all positions and total occurrences.

Program

```
def all_occurrences(arr, key):
    positions = []
```

```
for i in range(len(arr)):
    if arr[i] == key:
        positions.append(i + 1)

return positions

arr = [7, 12, 7, 25, 18, 7, 30, 7]
key = 7

pos = all_occurrences(arr, key)

print("Occurrences at positions:")
print(*pos, sep=" ")
print("Total occurrences =", len(pos))
```

Sample Input

Array: 7 12 7 25 18 7 30 7

Key: 7

Sample Output

Occurrences at positions:

1, 3, 6, 8

Total occurrences = 4

Result

Thus, all occurrences of the given element were found successfully.

Experiment 35: Sequential Search – Comparisons, Matches & Mismatches

Aim

To count total comparisons, matches and mismatches during Sequential Search.

Algorithm

1. Read the array and key.
2. Compare each element with the key.
3. Count comparisons.
4. Count matches and mismatches.
5. Display the counts.

Program

```
def search_statistics(arr, key):  
    comparisons = 0  
    matches = 0  
    mismatches = 0  
  
    for item in arr:  
        comparisons += 1  
        if item == key:  
            matches += 1  
        else:  
            mismatches += 1  
  
    print("Total Comparisons =", comparisons)  
    print("Total Matches =", matches)  
    print("Total Mismatches =", mismatches)  
  
arr = [3, 6, 9, 12, 15, 18, 21]  
key = 15
```

```
search_statistics(arr, key)
```

Sample Input

Array: 3 6 9 12 15 18 21

Key: 15

Sample Output

Total Comparisons = 7

Total Matches = 1

Total Mismatches = 6

Result

Thus, the total comparisons, matches, and mismatches were counted successfully using Sequential Search.

Experiment 36: Sentinel Sequential Search

Aim

To implement Sentinel Sequential Search and compare it with ordinary Sequential Search.

Algorithm

1. Read the array and search key.
2. Append the search key as a sentinel.
3. Search sequentially until the key is found.
4. If the index is within the original array, the element is found.
5. Otherwise, the element is not found.
6. Display the position and comparison count.

Program

```
def sentinel_search(arr, key):
```

```
    a = arr[:]
```



```
n = len(a)
```

```
a.append(key)
```

```
i = 0
```

```
comparisons = 0
```

```
while a[i] != key:
```

```
    comparisons += 1
```

```
    i += 1
```

```
comparisons += 1
```

```
if i < n:
```

```
    return i + 1, comparisons
```

```
else:
```

```
    return -1, comparisons
```

```
arr = [14, 9, 22, 35, 18, 41, 27]
```

```
key = 18
```

```
pos, comp = sentinel_search(arr, key)
```

```
if pos != -1:
```

```
    print("Position found:", pos)
```

```
else:
```

```
    print("Element not found")
```

```
print("Comparison count:", comp)
```

Sample Input

Array: 14 9 22 35 18 41 27

Key: 18

Sample Output

Position found: 5

Comparison count: 5

Result

Thus, Sentinel Sequential Search was implemented successfully.

Experiment 37: Sequential Search on Student Register Numbers

Aim

To search for a student register number using Sequential Search.

Algorithm

1. Read the register numbers.
2. Read the search key.
3. Compare each register number with the key.
4. Display the position if found.
5. Otherwise display "Not Found".

Program

```
def search_register(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i + 1  
    return -1
```

```
register = [101,102,103,104,105,106]
key = 104

pos = search_register(register, key)

if pos != -1:
    print("Register Number found at position", pos)
else:
    print("Register Number not found")
```

Sample Input

101 102 103 104 105 106

104

Sample Output

Register Number found at position 4

Result

Thus, the register number was found successfully using Sequential Search.

Experiment 38: Sequential Search for String Data**Aim**

To perform Sequential Search on string data.

Algorithm

1. Read the list of names.
2. Read the search name.
3. Compare each name with the search key.
4. Display the position if found.

Program

```
def string_search(names, key):  
    for i in range(len(names)):  
        if names[i] == key:  
            return i + 1  
    return -1  
  
names = ["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"]  
key = "Deepa"  
  
pos = string_search(names, key)  
  
if pos != -1:  
    print("Name found at position", pos)  
else:  
    print("Name not found")
```

Sample Input

Anu Bala Charan Deepa Esha Farhan

Deepa

Sample Output

Name found at position 4

Result

Thus, the string was searched successfully using Sequential Search.

Experiment 39: Sequential Search on a 2-D Matrix

Aim

To search an element in a two-dimensional matrix using Sequential Search.

Algorithm

1. Read the matrix.
2. Read the search key.
3. Traverse each row and column.
4. Compare every element.
5. Display the row and column if found.

Program

```
def matrix_search(matrix, key):
```

```
    for i in range(len(matrix)):
```

```
        for j in range(len(matrix[i])):
```

```
            if matrix[i][j] == key:
```

```
                return i + 1, j + 1
```

```
    return -1, -1
```

```
matrix = [
```

```
    [12,8,15],
```

```
    [5,18,27],
```

```
    [9,11,24]
```

```
]
```

```
key = 24
```

```
row, col = matrix_search(matrix, key)
```

```
if row != -1:
```

```
print("Element found at Row", row, "Column", col)
else:
    print("Element not found")
```

Sample Input

12 8 15

5 18 27

9 11 24

Key = 24

Sample Output

Element found at Row 3 Column 3

Result

Thus, the element was found successfully in the matrix using Sequential Search.

Experiment 40: Sequential Search Performance Analysis**Aim**

To analyze the performance of Sequential Search.

Algorithm

1. Read the array.
2. Read the search key.
3. Compare every element with the key.
4. Display each comparison.
5. Count comparisons.
6. Display best, average, worst case and complexity.

Program

```
def performance_search(arr, key):
```

```
comparisons = 0
```

```
for i in range(len(arr)):
```

```
    comparisons += 1
```

```
    print("Comparing", arr[i], "with", key)
```

```
    if arr[i] == key:
```

```
        print("Element found at position", i + 1)
```

```
        print("Comparisons =", comparisons)
```

```
    return
```

```
print("Element not found")
```

```
print("Comparisons =", comparisons)
```

```
arr = [45,23,67,12,89,34,56,78,90,11,29,73,18,64,37]
```

```
print("Search for 73")
```

```
performance_search(arr,73)
```

```
print("\nSearch for 18")
```

```
performance_search(arr,18)
```

```
print("\nSearch for 100")
```

```
performance_search(arr,100)
```

```
print("\nComplexity Analysis")
```

```
print("Best Case = O(1)")  
print("Average Case = O(n)")  
print("Worst Case = O(n)")  
print("Space Complexity = O(1)")
```

Sample Input

Array:

45 23 67 12 89 34 56 78 90 11 29 73 18 64 37

Keys:

73

18

100

Sample Output

Search for 73

Element found at position 12

Comparisons = 12

Search for 18

Element found at position 13

Comparisons = 13

Search for 100

Element not found

Comparisons = 15

Best Case = O(1)

Average Case = $O(n)$

Worst Case = $O(n)$

Space Complexity = $O(1)$

Result

Thus, Sequential Search was implemented successfully and its performance was analyzed.

Experiment 41: Brute Force String Matching – Find the First Occurrence

Aim

To implement the Brute Force String Matching algorithm to find the first occurrence of a pattern in a given text.

Algorithm

1. Read the text and pattern.
2. Compare the pattern with every possible position in the text.
3. If all characters match, display the starting position.
4. Otherwise continue searching.
5. If no match is found, display "Pattern not found."

Program

```
def brute_force(text, pattern):  
  
    n = len(text)  
    m = len(pattern)  
  
    for i in range(n - m + 1):  
  
        j = 0  
  
        while j < m and text[i + j] == pattern[j]:  
  
            j += 1  
  
        if j == m:
```

```
        return i

    return -1

text = "ALGORITHM"
pattern = "RITH"

pos = brute_force(text, pattern)

if pos != -1:
    print("Pattern found at index", pos)
else:
    print("Pattern not found")
```

Sample Input

Text = ALGORITHM

Pattern = RITH

Sample Output

Pattern found at index 4

Result

Thus, the Brute Force String Matching algorithm was implemented successfully.

Experiment 42: Brute Force String Matching – Find All Occurrences

Aim

To find all occurrences of a pattern in a text using the Brute Force String Matching algorithm.

Algorithm

1. Read the text and pattern.

2. Compare the pattern at every possible position.
3. Store every matching index.
4. Display all occurrences.

Program

```
def find_all(text, pattern):  
  
    n = len(text)  
  
    m = len(pattern)  
  
  
    found = False  
  
  
    for i in range(n - m + 1):  
  
        if text[i:i+m] == pattern:  
  
            print("Pattern found at index", i)  
  
            found = True  
  
  
    if not found:  
  
        print("Pattern not found")  
  
  
  
  
  
text = "ABABABABA"  
  
pattern = "ABA"  
  
  
  
  
  
find_all(text, pattern)
```

Sample Input

Text = ABABABABA

Pattern = ABA

Sample Output

Pattern found at index 0

Pattern found at index 2

Pattern found at index 4

Pattern found at index 6

Result

Thus, all occurrences of the pattern were found successfully.

Experiment 43: Brute Force String Matching – Comparison Table

Aim

To display the character comparisons performed during Brute Force String Matching.

Algorithm

1. Read the text and pattern.
2. Compare characters one by one.
3. Display every comparison.
4. Stop when the pattern is found.

Program

```
def compare(text, pattern):  
    n = len(text)  
    m = len(pattern)  
  
    for i in range(n - m + 1):  
        print("\nChecking position", i)  
  
        for j in range(m):  
            print(text[i + j], "=", pattern[j])
```

```
        if text[i + j] != pattern[j]:
            break

    else:

        print("Pattern found at index", i)

    return

text = "COMPUTER"

pattern = "PUT"

compare(text, pattern)
```

Sample Input

Text = COMPUTER

Pattern = PUT

Sample Output

Checking position 3

P == P

U == U

T == T

Pattern found at index 3

Result

Thus, the comparison process of Brute Force String Matching was displayed successfully.

Experiment 44: Brute Force String Matching – Count Comparisons

Aim

To count the total number of character comparisons during Brute Force String Matching.

Algorithm

1. Read the text and pattern.
2. Compare characters one by one.
3. Count every comparison.
4. Display the total comparisons.

Program

```
def comparison_count(text, pattern):  
    n = len(text)  
    m = len(pattern)  
  
    count = 0  
  
    for i in range(n - m + 1):  
        for j in range(m):  
            count += 1  
  
            if text[i + j] != pattern[j]:  
                break  
  
        else:  
            print("Pattern found at index", i)  
            break  
  
    print("Total comparisons =", count)  
  
text = "MISSISSIPPI"
```

```
pattern = "ISS"
```

```
comparison_count(text, pattern)
```

Sample Input

```
Text = MISSISSIPPI
```

```
Pattern = ISS
```

Sample Output

```
Pattern found at index 1
```

```
Total comparisons = 4
```

Result

Thus, the total number of comparisons was counted successfully.

Experiment 45: Brute Force String Matching – Complexity Analysis

Aim

To analyze the Best Case, Average Case, and Worst Case complexity of the Brute Force String Matching algorithm.

Algorithm

1. Read the text and pattern.
2. Search for the pattern using Brute Force.
3. Display whether the pattern exists.
4. Display the time complexities.

Program

```
def brute_force(text, pattern):
```

```
    n = len(text)
```

```
    m = len(pattern)
```

```
    for i in range(n - m + 1):
```

```
    if text[i:i+m] == pattern:  
        print("Pattern found at index", i)  
        return
```

```
print("Pattern not found")
```

```
text = "DATASTRUCTURES"
```

```
pattern = "STRUCT"
```

```
brute_force(text, pattern)
```

```
print("\nComplexity Analysis")
```

```
print("Best Case   : O(1)")
```

```
print("Average Case : O(nm)")
```

```
print("Worst Case  : O(nm)")
```

```
print("Space Complexity : O(1)")
```

Sample Input

```
Text = DATASTRUCTURES
```

```
Pattern = STRUCT
```

Sample Output

```
Pattern found at index 4
```

Complexity Analysis

Best Case : O(1)

Average Case : O(nm)

Worst Case : O(nm)

Space Complexity : $O(1)$

Result

Thus, the Brute Force String Matching algorithm was implemented successfully, and its time and space complexities were analyzed.

Experiment 46: Closest Pair of Points Using Brute Force

Aim

To find the closest pair of points in a set of 2D points using the brute force approach.

Algorithm

1. Read the list of points.
2. Calculate the Euclidean distance between every pair of points.
3. Keep track of the minimum distance.
4. Display the closest pair and minimum distance.

Program

```
import math

def distance(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

points = [(1,2), (4,5), (7,8), (3,1)]

min_dist = float('inf')
pair = ()

for i in range(len(points)):
    for j in range(i+1, len(points)):
        d = distance(points[i], points[j])
```

```
if d < min_dist:
    min_dist = d
    pair = (points[i], points[j])
```

```
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", min_dist)
```

Sample Input

Points: [(1,2), (4,5), (7,8), (3,1)]

Sample Output

Closest pair: (1, 2) - (3, 1)

Minimum distance: 2.23606797749979

Result

Thus, the closest pair of points was found successfully using the brute force approach.

Experiment 47: Closest Pair Using Exhaustive Comparison

Aim

To find the closest pair of points by comparing every possible pair.

Algorithm

1. Read the coordinates.
2. Compare every pair of points.
3. Compute Euclidean distance.
4. Display the closest pair.

Program

```
import math
```

```
points = [(2,3), (5,4), (1,7), (8,9), (4,6)]
```

```
minimum = float('inf')
pair = ()

for i in range(len(points)):
    for j in range(i+1, len(points)):
        d = math.dist(points[i], points[j])
        if d < minimum:
            minimum = d
            pair = (points[i], points[j])

print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", minimum)
```

Sample Input

Points: [(2,3), (5,4), (1,7), (8,9), (4,6)]

Sample Output

Closest pair: (5, 4) - (4, 6)

Minimum distance: 2.23606797749979

Result

Thus, the closest pair was identified successfully.

Experiment 48: Shortest Distance Between Any Two Points

Aim

To calculate the shortest distance between any two points using the brute force closest pair algorithm.

Algorithm

1. Read all points.

2. Compare each pair.
3. Find the minimum distance.
4. Display the closest pair.

Program

```
import math

points = [(0,0), (2,2), (3,1), (6,5)]

minimum = float('inf')
pair = ()

for i in range(len(points)):
    for j in range(i+1, len(points)):
        d = math.dist(points[i], points[j])
        if d < minimum:
            minimum = d
            pair = (points[i], points[j])

print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", minimum)
```

Sample Input

Points: [(0,0), (2,2), (3,1), (6,5)]

Sample Output

Closest pair: (2, 2) - (3, 1)

Minimum distance: 1.4142135623730951

Result

Thus, the shortest distance between two points was calculated successfully.

Experiment 49: Closest Pair of 2D Points

Aim

To identify the closest pair of points from a given collection of 2D points using the brute force technique.

Algorithm

1. Read the points.
2. Compare all possible pairs.
3. Find the minimum distance.
4. Display the closest pair.

Program

```
import math

points = [(10,20), (15,25), (30,40), (12,22)]

minimum = float('inf')
pair = ()

for i in range(len(points)):
    for j in range(i+1, len(points)):
        d = math.dist(points[i], points[j])
        if d < minimum:
            minimum = d
            pair = (points[i], points[j])

print("Closest pair:", pair[0], "-", pair[1])
```

```
print("Minimum distance:", minimum)
```

Sample Input

Points: [(10,20), (15,25), (30,40), (12,22)]

Sample Output

Closest pair: (10,20) - (12,22)

Minimum distance: 2.8284271247461903

Result

Thus, the closest pair of 2D points was identified successfully.

Experiment 50: Closest Pair with Time Complexity Analysis

Aim

To find the closest pair of points using the brute force approach and analyze its time complexity.

Algorithm

1. Define a function to compute Euclidean distance.
2. Compare every pair of points.
3. Store the minimum distance.
4. Display the closest pair.
5. Display the time complexity.

Program

```
import math
```

```
def distance(p1, p2):  
    return math.dist(p1, p2)
```

```
def closest_pair(points):  
    minimum = float('inf')
```

```
pair = ()
```

```
for i in range(len(points)):
```

```
    for j in range(i + 1, len(points)):
```

```
        d = distance(points[i], points[j])
```

```
        if d < minimum:
```

```
            minimum = d
```

```
            pair = (points[i], points[j])
```

```
return pair, minimum
```

```
points = [(10,0), (11,5), (5,3), (9,3.5),  
          (15,3), (12.5,7), (6,6.5), (7.5,4.5)]
```

```
pair, dist = closest_pair(points)
```

```
print("Closest pair:", pair[0], "-", pair[1])
```

```
print("Minimum distance:", dist)
```

```
print("\nTime Complexity : O(n2)")
```

```
print("Space Complexity : O(1)")
```

Sample Input

Points:

(10,0), (11,5), (5,3), (9,3.5),

(15,3), (12.5,7), (6,6.5), (7.5,4.5)

Sample Output

Closest pair: (9, 3.5) - (7.5, 4.5)

Minimum distance: 1.8027756377319946

Time Complexity : $O(n^2)$

Space Complexity : $O(1)$

Result

Thus, the closest pair of points was obtained successfully using the brute force method, and its time complexity was analyzed.

Experiment 51: Convex Hull Using Basic Method

Aim

To find the convex hull of a set of 2D points using the basic (brute force) method.

Algorithm

1. Read the set of points.
2. Sort the points.
3. Construct the lower hull.
4. Construct the upper hull.
5. Combine both hulls.
6. Display the convex hull points.

Program

```
def cross(o, a, b):  
    return (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])  
  
def convex_hull(points):  
    points = sorted(set(points))
```



```

if len(points) <= 1:
    return points

lower = []
for p in points:
    while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
        lower.pop()
    lower.append(p)

upper = []
for p in reversed(points):
    while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
        upper.pop()
    upper.append(p)

return lower[:-1] + upper[:-1]

```

```

points = [(0,0), (1,1), (2,2), (0,2), (2,0)]

```

```

print("Convex Hull Points:")
for p in convex_hull(points):
    print(p)

```

Sample Input

Points:

(0,0), (1,1), (2,2), (0,2), (2,0)

Sample Output

Convex Hull Points:

(0,0)

(2,0)

(2,2)

(0,2)

Result

Thus, the convex hull was found successfully.

Experiment 52: Boundary Points Forming Convex Hull

Aim

To determine the boundary points forming the convex hull.

Algorithm

1. Read the points.
2. Apply the convex hull algorithm.
3. Display the boundary points.

Program

```
from scipy.spatial import ConvexHull
```

```
points = [(1,2), (3,1), (5,3), (4,6), (2,5)]
```

```
hull = ConvexHull(points)
```

```
print("Convex Hull Points:")
```

```
for i in hull.vertices:
```

```
    print(points[i])
```

Sample Input

(1,2), (3,1), (5,3), (4,6), (2,5)

Sample Output

Convex Hull Points:

(1,2)

(3,1)

(5,3)

(4,6)

(2,5)

Result

Thus, the boundary points forming the convex hull were determined successfully.

Experiment 53: Convex Hull Using Brute Force

Aim

To generate the convex hull for a given set of coordinates using the brute force/basic method.

Algorithm

1. Read the coordinates.
2. Apply the convex hull algorithm.
3. Display the hull points.

Program

```
from scipy.spatial import ConvexHull
```

```
points = [(0,3), (1,1), (2,2), (4,4), (3,0)]
```

```
hull = ConvexHull(points)
```

```
print("Convex Hull Points:")
```

```
for i in hull.vertices:
```

```
    print(points[i])
```

Sample Input

(0,3), (1,1), (2,2), (4,4), (3,0)

Sample Output

Convex Hull Points:

(3,0)

(4,4)

(0,3)

Result

Thus, the convex hull was generated successfully.

Experiment 54: Outer Boundary Using Convex Hull

Aim

To find the outer boundary of a set of points using the convex hull algorithm.

Algorithm

1. Read the points.
2. Compute the convex hull.
3. Display the outer boundary points.

Program

```
from scipy.spatial import ConvexHull
```

```
points = [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]
```

```
hull = ConvexHull(points)
```

```
print("Convex Hull Points:")
```

```
for i in hull.vertices:
```

```
    print(points[i])
```

Sample Input

(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)

Sample Output

Convex Hull Points:

(4,1)

(6,3)

(5,5)

(3,6)

(1,4)

(2,2)

Result

Thus, the outer boundary of the given points was found successfully.

Experiment 55: Convex Hull Polygon Boundary

Aim

To calculate the convex hull of a given set of points and display the polygon boundary.

Algorithm

1. Read the points.
2. Compute the convex hull.
3. Display the polygon boundary.

Program

```
from scipy.spatial import ConvexHull
```

```
points = [(1,1), (2,4), (5,5), (6,2), (3,0)]
```

```
hull = ConvexHull(points)
```

```
print("Convex Hull Points:")
```

```
for i in hull.vertices:
```

```
    print(points[i])
```

Sample Input

(1,1), (2,4), (5,5), (6,2), (3,0)

Sample Output

Convex Hull Points:

(3,0)

(6,2)

(5,5)

(2,4)

Result

Thus, the convex hull polygon boundary was obtained successfully.

Experiment 56: Exhaustive Search for Subset Sum

Aim

To determine whether a subset exists with a given sum using the exhaustive search technique.

Algorithm

1. Read the set of elements and the target sum.
2. Generate all possible subsets.
3. Calculate the sum of each subset.

4. If a subset equals the target sum, display it.
5. Otherwise, display that no subset exists.

Program

```
from itertools import combinations
```

```
def subset_sum(arr, target):
```

```
    n = len(arr)
```

```
    for r in range(1, n + 1):
```

```
        for subset in combinations(arr, r):
```

```
            if sum(subset) == target:
```

```
                return list(subset)
```

```
    return None
```

```
arr = [3, 34, 4, 12, 5, 2]
```

```
target = 9
```

```
result = subset_sum(arr, target)
```

```
if result:
```

```
    print("Subset found:", result)
```

```
    print("Sum:", sum(result))
```

```
else:
```

```
    print("No subset found")
```

Sample Input

Set: [3, 34, 4, 12, 5, 2]

Target Sum: 9

Sample Output

Subset found: [4, 5]

Sum: 9

Result

Thus, the subset with the given sum was found successfully using exhaustive search.

Experiment 57: Travelling Salesman Problem Using Exhaustive Search

Aim

To find the shortest path for the Travelling Salesman Problem using exhaustive search.

Algorithm

1. Read the coordinates of the cities.
2. Generate all possible tours.
3. Compute the total distance of each tour.
4. Select the tour with the minimum distance.
5. Display the shortest distance and path.

Program

```
from itertools import permutations
```

```
import math
```

```
cities = [(1,2), (4,5), (7,1), (3,6)]
```

```
def distance(a, b):
```

```
    return math.dist(a, b)
```



```

best_path = None
best_distance = float('inf')

for perm in permutations(cities[1:]):
    path = [cities[0]] + list(perm) + [cities[0]]

    total = 0

    for i in range(len(path)-1):
        total += distance(path[i], path[i+1])

    if total < best_distance:
        best_distance = total
        best_path = path

print("Shortest Distance:", best_distance)
print("Shortest Path:", best_path)

```

Sample Input

Cities:

```
[(1,2), (4,5), (7,1), (3,6)]
```

Sample Output

Shortest Distance: 18.69398062518129

Shortest Path:

```
[(1,2), (4,5), (3,6), (7,1), (1,2)]
```

Result

Thus, the shortest travelling path was obtained successfully using exhaustive search.

Experiment 58: Assignment Problem Using Exhaustive Search

Aim

To solve the Assignment Problem using exhaustive search.

Algorithm

1. Read the cost matrix.
2. Generate all possible assignments.
3. Compute the total cost of each assignment.
4. Select the assignment with the minimum cost.
5. Display the optimal assignment.

Program

```
from itertools import permutations
```

```
cost = [  
    [3,10,7],  
    [8,5,12],  
    [4,6,9]  
]
```

```
n = len(cost)
```

```
best_cost = float('inf')
```

```
best_assign = None
```

```
for perm in permutations(range(n)):
```

```
    total = 0
```

```
for worker in range(n):  
    total += cost[worker][perm[worker]]  
  
    if total < best_cost:  
        best_cost = total  
        best_assign = perm  
  
print("Optimal Assignment:")  
for i in range(n):  
    print("Worker", i+1, "-> Task", best_assign[i]+1)  
  
print("Total Cost:", best_cost)
```

Sample Input

Cost Matrix:

3 10 7

8 5 12

4 6 9

Sample Output

Optimal Assignment:

Worker 1 -> Task 3

Worker 2 -> Task 2

Worker 3 -> Task 1

Total Cost: 16

Result

Thus, the Assignment Problem was solved successfully using exhaustive search.

Experiment 59: 0–1 Knapsack Using Exhaustive Search

Aim

To solve the 0–1 Knapsack Problem using exhaustive search.

Algorithm

1. Read the weights, values and capacity.
2. Generate all subsets.
3. Check whether each subset satisfies the capacity.
4. Compute the total value.
5. Select the subset with the maximum value.

Program

```
from itertools import combinations

weights = [2,3,1]
values = [4,5,3]
capacity = 4

best_value = 0
best_items = []

n = len(weights)

for r in range(n+1):
    for subset in combinations(range(n), r):
        total_weight = sum(weights[i] for i in subset)
        total_value = sum(values[i] for i in subset)
```

```
if total_weight <= capacity and total_value > best_value:
```

```
    best_value = total_value
```

```
    best_items = list(subset)
```

```
print("Optimal Selection:", best_items)
```

```
print("Total Value:", best_value)
```

Sample Input

Weights: [2,3,1]

Values: [4,5,3]

Capacity: 4

Sample Output

Optimal Selection: [0, 2]

Total Value: 7

Result

Thus, the optimal solution of the 0–1 Knapsack Problem was obtained successfully.

Experiment 60: Subset Sum by Generating All Subsets

Aim

To solve the subset sum problem by generating all possible subsets.

Algorithm

1. Read the set and target sum.
2. Generate every possible subset.
3. Compute the sum of each subset.
4. If a subset equals the target sum, display it.

Program

```
from itertools import combinations
```

```
arr = [15,10,12,7,5]
```

```
target = 22
```

```
found = False
```

```
for r in range(1, len(arr)+1):
```

```
    for subset in combinations(arr, r):
```

```
        if sum(subset) == target:
```

```
            print("Subset found:", list(subset))
```

```
            print("Sum:", sum(subset))
```

```
            found = True
```

```
            break
```

```
if found:
```

```
    break
```

if not found:

```
print("No subset found")
```

Sample Input

Set: [15,10,12,7,5]

Target Sum: 22

Sample Output

Subset found: [15, 7]

Sum: 22

Result

Thus, the subset with the required sum was found successfully by generating all possible subsets.